

DISPENSA DELLA LEZIONE

Eventi, Hook e stato locale

Gestione delle interazioni e stato reattivo nei componenti React.



CAPITOLO 1

Gestione eventi in React

NOTA

Un evento è una azione compiuta dall'utente mentre usa la pagina, come un click su un bottone o la scrittura in un campo di testo. In React si gestisce assegnando una funzione a una prop speciale, come `onClick`.

Ogni elemento JSX può reagire a un evento tramite una prop dedicata. La prop più usata è `onClick`, che si applica a un elemento come un bottone e indica quale funzione eseguire quando quell'elemento viene cliccato.

Il valore della prop deve essere il nome della funzione, senza le parentesi tonde: se si scrivono le parentesi, la funzione viene eseguita subito durante il rendering, invece che al momento del click. A volte però serve proprio eseguire subito una piccola operazione al click, come passare un valore diverso a una funzione: in quel caso si scrive una funzione freccia dentro `onClick`, così il codice al suo interno parte solo quando l'utente clicca davvero.

Esempio

```
function BottoneMessaggio() {  
  let numero = 0;  
  
  function saluta() {  
    console.log("Click ricevuto");  
  }  
  
  function incrementa(valore) {  
    numero = numero + valore;  
    console.log("Numero attuale:", numero);  
  }  
  
  return (  
    <div>  
      // Corretto: la funzione viene passata, non eseguita  
      <button onClick={saluta}>  
        Premi qui  
      </button>  
  
      // Sbagliato: la funzione verrebbe eseguita subito  
      // <button onClick={saluta()}>  
      //   Premi qui  
      // </button>  
  
      // Funzione freccia: incrementa di 1 solo al click  
      <button onClick={() => incrementa(1)}>  
        Incrementa  
      </button>  
    </div>  
  );  
}
```

CAPITOLO 2

Eventi più comuni

NOTA

React mette a disposizione molte prop evento diverse. Le più usate servono a gestire click sui bottoni, scrittura nei campi di testo, invio dei form, focus della tastiera e passaggio del mouse sopra un elemento.

Ogni evento è pensato per un tipo di interazione precisa. Un bottone tipicamente reagisce a `onClick`, un campo di input reagisce a `onChange` mentre l'utente digita, e un form intero reagisce a `onSubmit` quando viene inviato.

Conoscere questi eventi permette di scegliere quello corretto in base a cosa deve succedere nella pagina. Un errore comune è usare `onClick` per intercettare la scrittura in un campo di testo, quando in realtà per quel caso serve `onChange`.

Evento	Descrizione	Elementi comuni
<code>onClick</code>	L'utente preme e rilascia il mouse su un elemento.	button, a, div
<code>onSubmit</code>	Viene inviato un form.	form
<code>onChange</code>	Il valore di un campo cambia.	input, select, textarea
<code>onInput</code>	Il valore viene modificato durante la digitazione.	input, textarea
<code>onFocus</code>	Un elemento riceve il focus.	input, button, a
<code>onBlur</code>	Un elemento perde il focus.	input, button, a
<code>onKeyDown</code>	L'utente preme un tasto sulla tastiera.	input, document
<code>onKeyUp</code>	L'utente rilascia un tasto sulla tastiera.	input, document
<code>onMouseOver</code>	Il puntatore entra in un elemento.	div, button, img
<code>onMouseOut</code>	Il puntatore esce da un elemento.	div, button, img

Esempio

```
function CampoNome() {  
  function scriviInConsole() {  
    console.log("Valore modificato");  
  }  
  
  function inviaForm() {  
    console.log("Form inviato");  
  }  
  
  return (  
    <form onSubmit={inviaForm}>  
      <input type="text" onChange={scriviInConsole} />  
      <button>Invia</button>  
    </form>  
  );  
}
```

CAPITOLO 3

Che cos'è un Hook

NOTA

Un Hook è una funzione speciale che permette a un componente funzione di "agganciarsi" a funzionalità interne di React, come la memorizzazione di uno stato.

Un componente funzione, di per sé, esegue il proprio codice e restituisce del JSX, ma ogni volta che viene rieseguito perde le variabili dichiarate al suo interno con `let` o `const`. Questo significa che un valore come un contatore non potrebbe essere ricordato tra un click e l'altro senza uno strumento apposito.

Gli Hook risolvono questo problema: sono funzioni fornite da React che, se chiamate dentro un componente, permettono di conservare valori tra un rendering e l'altro e di collegare il componente ad altre funzionalità di React. Il nome di ogni Hook inizia sempre con il prefisso `use`, come `useState`: questo rende immediatamente riconoscibile che si tratta di un Hook e non di una funzione qualsiasi.

Esempio

```
import { useState } from "react";

function EsempioHook() {
  const [valore, setValore] = useState(0);

  return (
    <p>Valore: {valore}</p>
  );
}
```

CAPITOLO 4

L'Hook useState

NOTA

`useState` è l'Hook che permette di aggiungere una variabile di stato a un componente. Restituisce sempre due valori: lo stato attuale e la funzione che serve per aggiornarlo.

`useState` si importa dal pacchetto `react` e si chiama all'inizio del componente, passando come argomento il valore iniziale dello stato. Questo valore iniziale viene usato solo la prima volta che il componente viene creato.

La funzione restituisce un array con due elementi, che di solito si estraggono con la destrutturazione: il primo elemento è il valore corrente dello stato, il secondo è la funzione da chiamare per cambiarlo, per convenzione chiamata `setNomeStato`. Lo stato non va mai modificato direttamente assegnando un nuovo valore alla variabile: bisogna sempre usare la funzione di aggiornamento, altrimenti React non si accorge del cambiamento e non aggiorna la pagina.

Ogni volta che la funzione di aggiornamento viene chiamata con un valore diverso da quello attuale, React riesegue il componente usando il nuovo valore, così l'interfaccia mostra sempre il dato aggiornato.

Esempio

```
import { useState } from "react";

function Contatore() {
  const [count, setCount] = useState(0);

  function incrementa() {
    setCount(count + 1);
  }

  return (
    <div>
      <p>Conteggio: {count}</p>
      <button onClick={incrementa}>
        Aggiungi
      </button>
    </div>
  );
}
```

CAPITOLO 5

Reattività e aggiornamento UI

NOTA

La reattività è il comportamento per cui l'interfaccia si aggiorna automaticamente quando cambia uno stato, senza bisogno di ricaricare la pagina.

Quando la funzione di aggiornamento dello stato viene chiamata, React confronta il nuovo valore con quello precedente e, se sono diversi, riesegue il componente per calcolare il nuovo contenuto da mostrare. Questo collega direttamente lo stato a tutto ciò che dipende da esso nel JSX, come testo, numeri o blocchi mostrati con il conditional rendering.

Uno stato booleano, per esempio, può decidere se un blocco di JSX viene mostrato oppure no grazie allo short-circuiting con l'operatore `&&`. Ogni volta che lo stato booleano cambia valore, il blocco condizionale appare o scompare automaticamente, senza altro codice aggiuntivo.

Esempio

```
import { useState } from "react";

function MessaggioVisibile() {
  const [visibile, setVisibile] = useState(false);

  function cambiaStato() {
    setVisibile(!visibile);
  }

  return (
    <div>
      <button onClick={cambiaStato}>
        Cambia stato
      </button>

      {visibile && <p>Messaggio visibile</p>}
    </div>
  );
}
```